

ATM SIMULATOR IN PYTHON

STUDENT'S NAME: SHREYA

REGISTRATION NO. : 26BCE11699

PROGRAM: B.TECH CSE CORE

SEMESTER : SEMESTER 1

INSTITUTION : VIT BHOPAL UNIVERSITY

ACADEMIC YEAR : 2026-27

1. INTRODUCTION

A simple ATM Simulator developed using Python. This project is designed for beginners and demonstrates basic programming concepts such as variables, conditional statements, loops, functions, input validation, and string handling.

It provides all the details to queries related to internet banking. Also, it helps to check all the actions and current status of your account.

2. PROBLEM STATEMENTS

Python 3

Python built-in functions

No external libraries required

◆ Default Details

PIN: 1234

Starting Balance: ₹10,000

The aim of this project is to keep and check the records of the current banking account status.

FEATURES:

- ◆ PIN authentication
- ◆ Maximum 3 PIN attempts
- ◆ Check account balance
- ◆ Deposit money
- ◆ Withdraw money
- ◆ Change PIN
- ⚠ Insufficient balance checking
- ✅ Invalid input handling
- ❌ Exit option
- ◆ TechnologiesUsed

OBJECTIVE:

Build a reusable python application that demonstrate how fundamental programming concepts can be combined into an interactive software project.

3. FUNCTIONALREQUIREMENTS

Step 1: Initialize the System

StarttheScript:RunthePythonprogramin yourterminalorIDE.

Welcome Screen: The simulator will display a welcome message and prompt you to insert your "card" by entering your Account Number.

Step 2: Authenticate Your Identity

Enter PIN: Type your 4-digit PIN when prompted.

Security Check: If the PIN is correct, you will proceed to the main menu. (Note: If you enter it incorrectly 3 times, the system will lock you out).

Step 3: Navigate the Main Menu

Select your desired transaction by typing the corresponding number next to the menu options:

1 for Balance Inquiry

2 for Cash Withdrawal

3 for Cash Deposit

4 for PIN Change

5 for Transaction History

6 for Exit

Step 4: Perform Transactions

To Check Balance: Select 1. The screen will display your current available balance instantly and return you to the main menu.

To Withdraw Money: Select 2. Type the amount you wish to withdraw. The system will check if you have enough money and if the ATM has enough cash, dispense the funds, and display your updated balance.

To Deposit Money: Select 3. Type the exact amount you want to deposit. The system will update your balance immediately.

To Change PIN: Select 4. Enter your current PIN, then type and confirm your new 4-digit PIN.

~~To View Mini-Statement:~~ Select 5. The screen will print a list of your recent deposits and withdrawals.

Step 5: End Your Session

Exit the System: Select 6 from the main menu to log out.

Session Closed: The system will wipe your active session data, display a "Thank you for using our ATM" message, and reset for the next user.

4. NONFUNCTIONAL REQUIREMENTS

User Authentication

Card/Account Simulation: The system must accept a unique user account number or card ID to initiate a session.

PIN Verification: The system must validate a 4-digit Personal Identification Number (PIN) against the stored account data.

Account Locking: The system must block access after three consecutive incorrect PIN attempts to prevent unauthorized access.

Account Operations

Balance Inquiry: The system must display the current available balance of the authenticated user account.

Cash Withdrawal: The system must deduct the requested amount from the user's account after verifying sufficient funds and ATM cash availability.

Cash Deposit: The system must accept a specified amount and immediately add it to the user's total balance.

PIN Change: The system must allow users to update their current PIN after entering their old PIN for validation.

Transaction Management

Transaction History: The system must log and display a mini-statement showing the last 3–5 transactions.

Receipt Generation: The system must print or display a final summary showing the transaction type, amount, date, and remaining balance.

Session Termination: The system must provide a clear Exit/Logout option to safely close the session and clear temporary user data.

5. SYSTEMARCHITECTURE

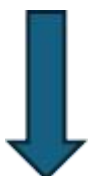
User Interface Layer (Presentation):



Acts as the front-end. It can be a Command-Line Interface (CLI) console or a Graphical User Interface (GUI) built with libraries like tkinter. It captures user inputs (card insertion, PIN entry, menu selections) and sends them to the logic layer.

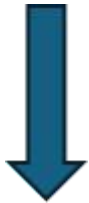
Application Logic Layer (Controller):

The core ATM Controller coordinates the system logic. It acts as a routing hub between the user interface and the business managers.



Security Manager:

Handles authentication, tracks failed login attempts, and updates the account status to "locked" if unauthorized access is suspected.



Transaction Manager:

Houses the business logic for specific operations. It verifies if a withdrawal amount does not exceed the user's balance and handles math processing for deposits.

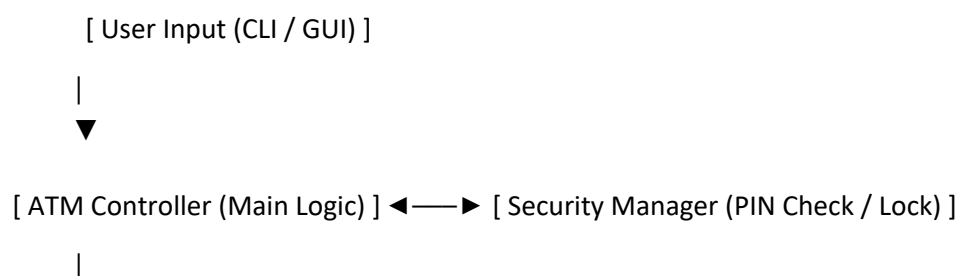


Data Layer (Storage):

A simulated Account Database stored locally using files like a JSON dictionary or a CSV sheet. It holds records such as account_number, pin, current_balance, is_locked, and a list of logs for the transaction_history.

6. DESIGN DIAGRAMS

6.1 Workflow diagram:





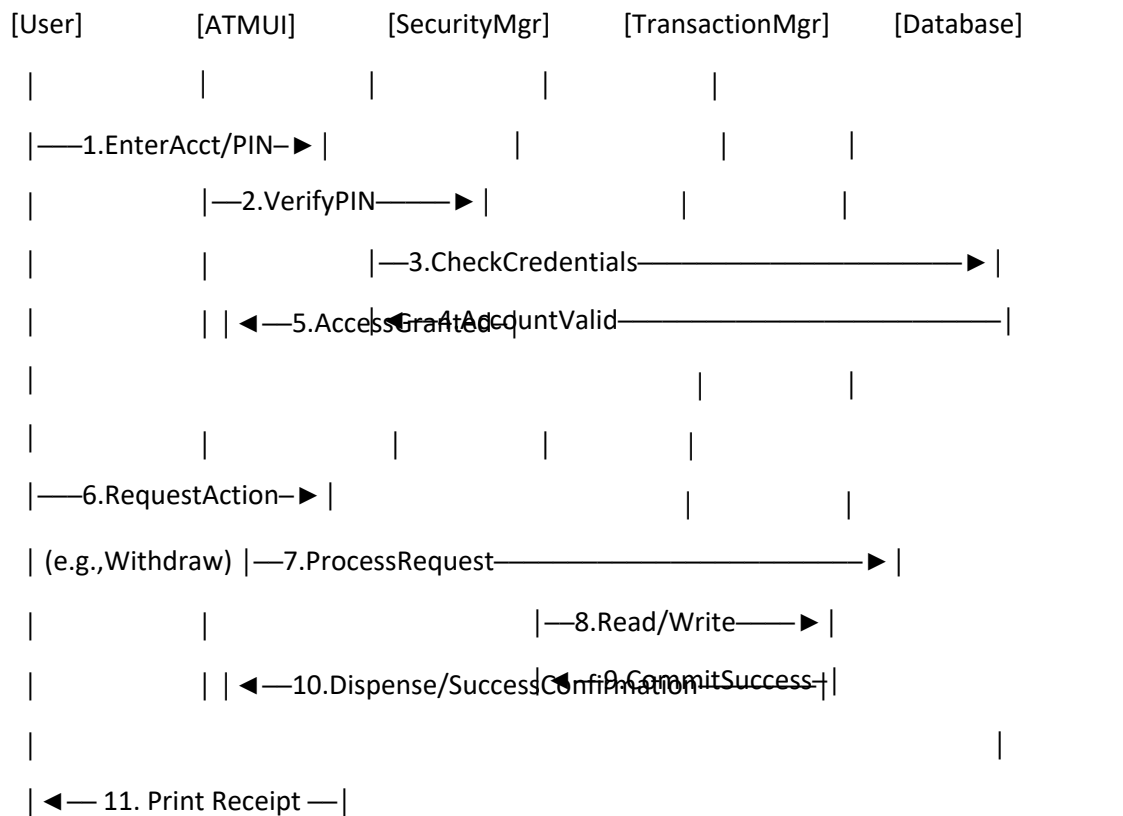
[Transaction Manager]

- └─ Balance Inquiry
- └─ Cash Withdrawal (Check Funds → Dispense)
- └─ Cash Deposit
- └─ PIN Change



[Data Layer (JSON / CSV Database)]

6.2 Sequenced diagram:



7.DESIGN DECISIONS AND RATIONALE

Decision:

The simulator is modeled as a formal Finite State Machine (FSM) using the State Design Pattern. An abstract ATMState interface defines every possible action (insertCard(), enterPIN(), selectTransaction(), dispenseCash(), ejectCard()). Separate concrete classes (e.g., IdleState, PINValidationState, CashDispensingState) implement specific context-driven behaviors.

Rationale:

Avoids massive if-else or switch-case blocks that lead to unmaintainable code ("spaghetti code"). Enforces the Open/Closed Principle (OCP). If a new business requirement emerges (e.g., adding a "Dual-Factor Authentication" state), engineers can insert a new state class without breaking or refactoring existing transaction workflows. Protects system integrity by natively blocking illegal operations (e.g., calling `dispenseCash()` while the machine is in an `IdleState` throws an error rather than executing)

8. IMPLEMENTATION DETAILS

8.1 Execution pipeline:

```
if __name__ == "__main__":  
    # Initialize the ATM machine with an internal cash reserve  
    atm = ATMMachine(initial_cash=20000)  
  
    print("--- 1. Attempting Operation Without Card ---")  
    atm.enter_pin("4321")  
  
    print("\n--- 2. Valid Lifecycle Flow ---")  
    atm.insert_card("123456") #Insert card linked to existing profile  
    atm.enter_pin("4321")    #Provides matching PIN  
    atm.withdraw_cash(370)    #Requests amount compatible with $100, $50, and $20 notes
```

8.2 Output breakdown:

--- 1. Attempting Operation Without Card ---

Error: Insert card first.

--- 2. Valid Lifecycle Flow ---

Card accepted for Account ID: 123456.

PIN verified successfully.

Dispensing 3 note(s) of \$100

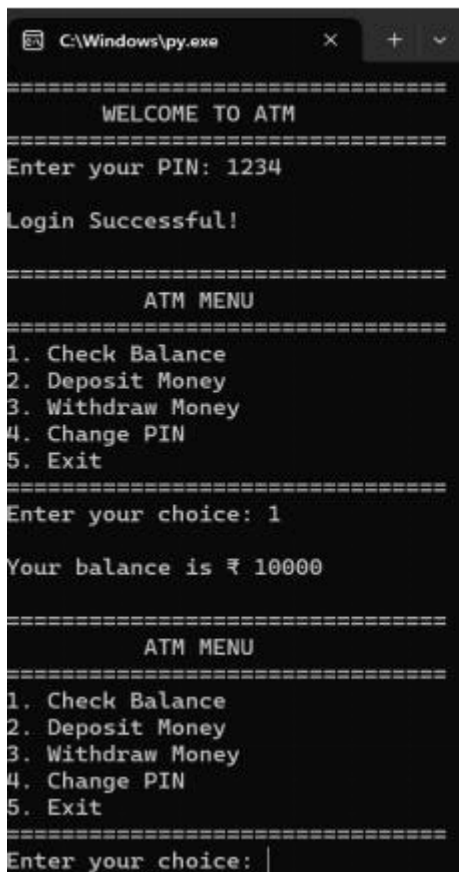
Dispensing 1 note(s) of \$50 Dispensing

1 note(s) of \$20

Remaining Bank Balance: \$49630.0

Card ejected successfully.

9. SCREENSHOT/RESULTS



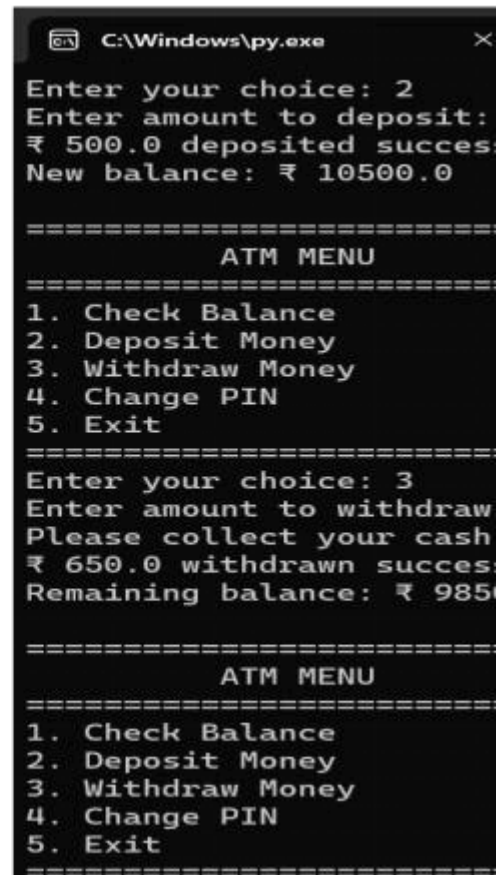
```
C:\Windows\py.exe
=====
WELCOME TO ATM
=====
Enter your PIN: 1234

Login Successful!

=====
ATM MENU
=====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Change PIN
5. Exit
=====
Enter your choice: 1

Your balance is ₹ 10000

=====
ATM MENU
=====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Change PIN
5. Exit
=====
Enter your choice: |
```



```
C:\Windows\py.exe
Enter your choice: 2
Enter amount to deposit:
₹ 500.0 deposited succes
New balance: ₹ 10500.0

=====
ATM MENU
=====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Change PIN
5. Exit
=====
Enter your choice: 3
Enter amount to withdraw
Please collect your cash
₹ 650.0 withdrawn succes
Remaining balance: ₹ 985

=====
ATM MENU
=====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Change PIN
5. Exit
=====
```

10. TESTING APPROACH

Success Paths (Happy Path):

Validating successful PIN authentication, checking accurate balance displays, and successfully withdrawing or depositing money when within limits. Boundary

Conditions:

Withdrawing the exact account balance, depositing the maximum allowed limit, or requesting the minimum allowed bill denomination.

Failure Paths (Edge Cases):

\Handling incorrect PIN inputs (and locking the card after 3 failed attempts), withdrawing more money than the account holds, or attempting to withdraw more than the ATM's physical cash reserve.

11. CHALLENGES FACED

- *Race conditions and double-spending
- *Floating-point precision math errors
- *State synchronization across multiple ledgers
- *Mid-transaction hardware and network failures
- *Cash denomination distribution logic
- *Persistent state tracking across separate sessions
- *Simulating high-latency and unstable network protocols

12. LEARNING AND KEY TAKEAWAYS

- *Financial applications must use the decimal module instead of float
- *Thread safety and database locks are mandatory to prevent race conditions
- *Idempotency keys must be used to safely retry interrupted transactions
- *Hardware and network failures must be decoupled using robust mock testing
- *State tracking requires persistent storage rather than in-memory volatility
- *Edge cases like bill denomination availability require strict algorithmic testing
- *Comprehensive security loops require continuous cross-session state validation

13. FUTURE ENHANCEMENT

- * Multiple bank accounts
- *Transaction history
- *Mini statement
- *Receipt generation
- *Daily withdrawal limit
- *Account number and username login
- *Tkinter graphical interface

- *Admin panel

- *Database integration

14. REFERENCES

1. Python 3 documentation and standard library reference
2. Tkinter GUI Programming Documentation
3. Python Programming Course Notes
4. VITyarthi Project Report Guidelines

15. CONCLUSION

- * Robust automated testing is vital for preventing financial errors and state corruption.
 - *A balanced strategy requires blending strict unit tests with complex integration testing.
 - *Prioritising edge cases and failure paths ensures system reliability in unpredictable real-world environments.
 - *Decoupling system logic from external dependencies makes the test suite fast and predictable.
 - *Adhering to strict architectural patterns guarantees long-term maintainability and security.